Resumen de IS2 para el Parcial 2

Hola xd

Unidad 4: Patrones orientados a Objetos

Qué son

Son soluciones comunes a problemas que ocurren con frecuencia en diseños de software. Es un molde a seguir para solucionar problemas recurrentes en el código.

Los patrones son conceptos generales que se aplican para resolver problemas, pero los haces encajar vos.

El patrón consiste de:

- Un propósito, explicado por un problema y una solución
- Una motivación
- Una estructura de clases que muestra las partes del patrón y como se relacionan.

Básicamente, son soluciones probadas y efectivas para problemas comunes en el diseño de software. Los patrones facilitan la creación de sistemas más flexibles, mantenibles y escalables.

Historia

te pensaste q iba a poner la historia re boludo mal, fue una idea loca de un chabón en un libro, que después la agarraron 4 pobres locos más y dijeron bueno acá estos son los patrones de diseño toma, y lo publicaron en un libro q tenía un título re largo por lo k la gente le puso el apodo re piola “Gang of Four” (*los cuatro mosqueteros*, para los que no saben inglés).

Ventajas

Sirven para comunicarse más eficientemente con el team de desarrollo, y para ahorrar tiempo al implementar soluciones.

Crítica

Acá el libro se pone a bardear, pero básicamente dice:

- Úsalo si el lenguaje lo permite, sino es al pedo.
- Si lo vas a usar, adáptalo a lo que necesitas, no copies y pegues el diseño a lo burro.
- Justifica su uso, no lo uses porque pinto o porque se ve piola.

Clasificación de patrones

Por nivel

Los básicos y de bajo nivel se suelen llamar “idioms”, suelen aplicarse en un único lenguaje de programación.

Los más universales y de alto nivel son los patrones de Arquitectura. Se pueden implementar en cualquier lenguaje, y sirven para diseñar la arquitectura de una aplicación completa.

Por propósito

Lo que usamos nosotros.

- **Creacionales:** Dan mecanismos para la creación de objetos, aumentan la flexibilidad y reutilización del código.
- **Estructurales:** Explican como estructurar objetos y clases en estructuras más grandes, manteniendo la flexibilidad y eficiencia de la estructura.
- **Comportamiento:** Se encargan de la comunicación efectiva y asignar responsabilidades entre los objetos.



Todos los patrones que hay

// No tengo idea si esto entra en el parcial así q copio y pego noma

1. Patrones creacionales:

   • Singleton: Garantiza que una clase solo tenga una única instancia y proporciona un punto de acceso global a ella.

   • Factory Method: Permite a una clase delegar la creación de objetos a sus subclases, sin especificar la clase concreta.

   • Abstract Factory: Proporciona una interfaz para crear familias de objetos relacionados sin especificar sus clases concretas.

   • Builder: Abstrae la construcción de un objeto complejo, permitiendo crear diferentes representaciones del mismo proceso de construcción.

   • Prototype: nos permite copiar objetos existentes sin que el código dependa de sus clases.

2. Patrones estructurales:

• Adapter: Permite que clases incompatibles trabajen juntas mediante la conversión de interfaces.

• Decorator: Agrega comportamiento adicional a objetos dinámicamente sin modificar su estructura.

• Proxy: Actúa como intermediario para controlar el acceso a un objeto y añadir funcionalidades adicionales.

• Composite: Permite tratar objetos individuales y composiciones de objetos de manera uniforme, tratándolos como una estructura jerárquica.

3. Patrones de comportamiento:

• Observer: Establece una dependencia uno-a-muchos entre objetos, de modo que cuando un objeto cambia de estado, todos los objetos dependientes son notificados y actualizados automáticamente.

• Strategy: Permite cambiar dinámicamente el algoritmo utilizado por un objeto en tiempo de ejecución.

• Command: Encapsula una solicitud como un objeto, lo que permite parametrizar clientes con diferentes solicitudes y soportar operaciones reversibles (deshacer).

• Template Method: Define el esqueleto de un algoritmo en una clase base, permitiendo que las subclases implementen ciertos pasos del algoritmo sin cambiar su estructura general.

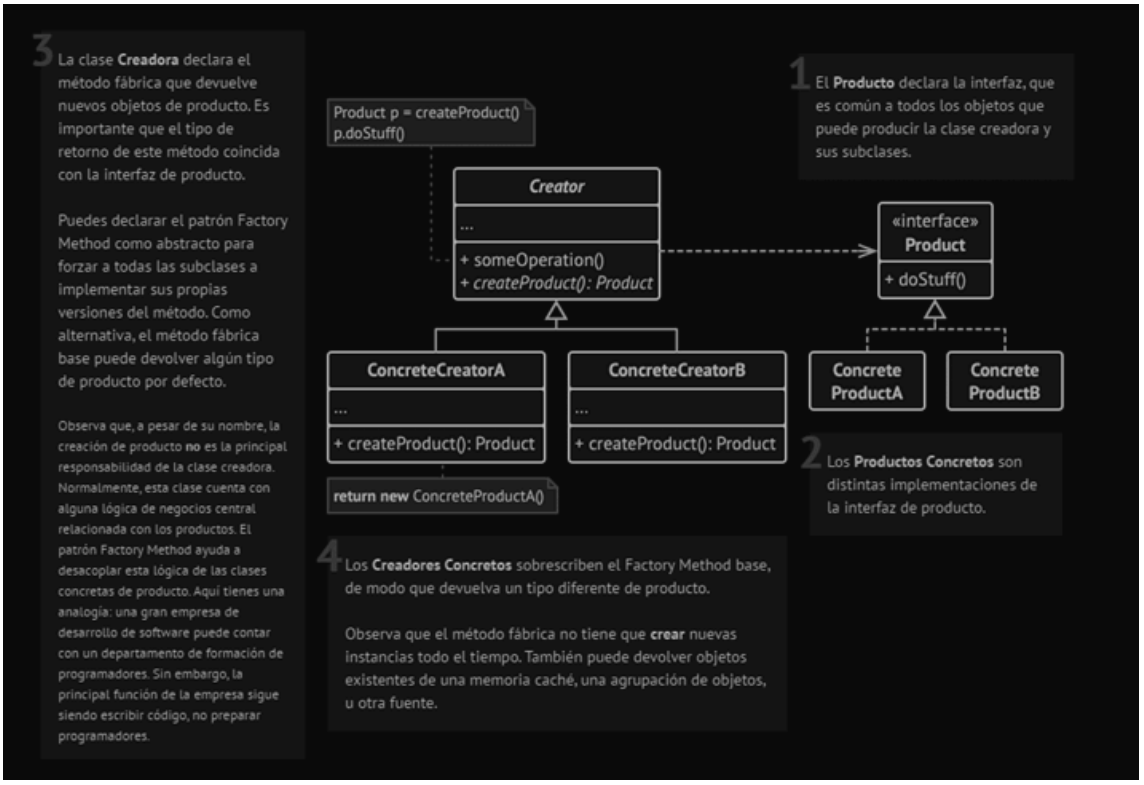
Patrones de diseño Creacionales

Factory Method

Propósito:

Proporciona una interfaz para crear objetos en una superclase, mientras permite a las subclases alterar el tipo de objetos que van a crear.

Sugiere que en vez de usar “new” para construir objetos, se invoque a un método **fábrica** especial. Por lo tanto, es la fábrica la que crea y devuelve estos objetos, los cuales se les suele llamar “productos”.



Aplicación

1. Se aplica cuando no conoces de antemano las dependencias y tipos exactos de los objetos con los que tiene que funcionar el código.
2. Cuando quieres ofrecer una biblioteca o framework, como una forma de extender sus componentes internos.
3. Para ahorrar recursos mediante la reutilización de objetos existentes en vez de volver a construirlos.

Pros/Contras

✔ Evita acoplamiento fuerte entre el creador y los productos concretos.

✔ Sigue el principio de responsabilidad única.

✔ Principio de abierto/cerrado, puede incorporar nuevos tipos de productos sin descomponer el código cliente existente (código que usa estos objetos).

⌦ El código se puede complicar porque hay que introducir varias nuevas subclases para implementar el patrón. Lo ideal es introducirlo en una jerarquía existente de clases creadoras.

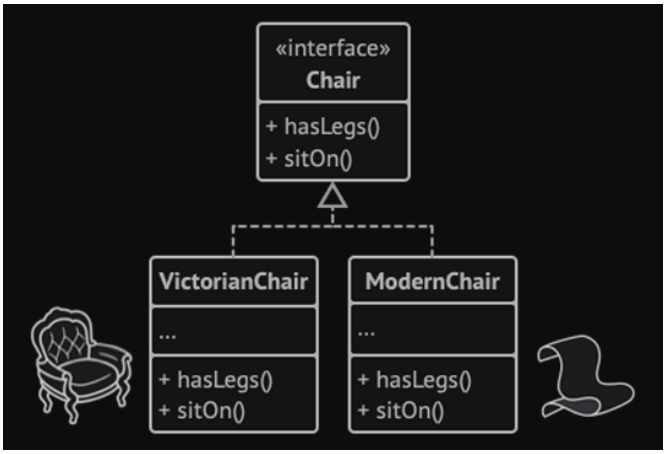
Abstract Factory

Propósito

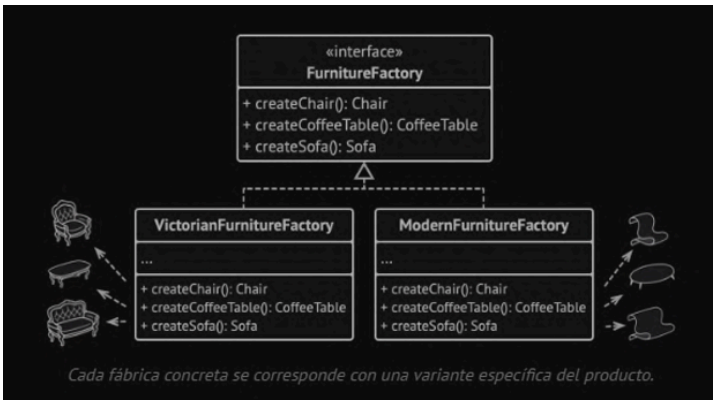
Permite producir familias de objetos relacionados sin especificar sus clases concretas.

Solución

Sugiere declarar de forma explícita interfaces para cada producto diferente de la familia de productos, para que cada variante de ese producto siga la misma interfaz.

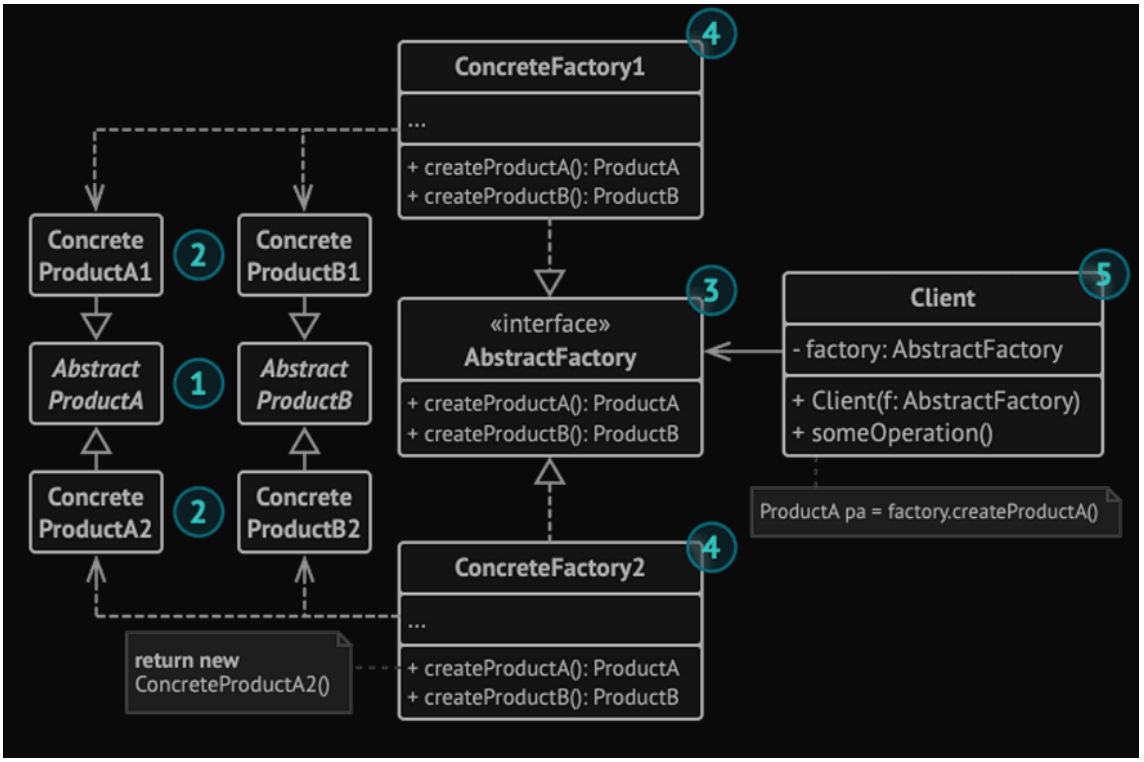


Luego declaramos la fábrica abstracta con una interfaz de métodos de creación para todos los productos pertenecientes a la familia de productos, donde se devuelven **productos abstractos** representados por las interfaces, como en el ejemplo la Silla.



Para crear cada producto especifico que necesitamos, es decir para crear una variante, creamos clases basadas en la interfaz abstracta, que sirvan como fábrica para crear dichos productos.

ESTRUCTURA



Aplicabilidad

- Útil para funcionar con familias de productos relacionados.
- Ofrece una interfaz para crear objetos a partir de cada clase de la familia de productos.
- Cuando tenes una clase con un grupo de métodos de fábrica que nublen su responsabilidad principal.
- Cada clase es responsable tan solo de una cosa.

Implementación

Normalmente no pongo esto, pero me parece interesante el concepto para aplicarlo en la práctica, **hay que mapear los tipos de productos y sus variantes como si fueran una matriz.**

Pros/Contras

✔ Puedes tener la certeza de que los productos que obtienes de una fábrica son compatibles entre sí.

✔ Evitas un acoplamiento fuerte entre productos concretos y el código cliente.

✔ Principio de responsabilidad única. Puedes mover el código de creación de productos a un solo lugar, haciendo que el código sea más fácil de mantener.

✔ Principio de abierto/cerrado. Puedes introducir nuevas variantes de productos sin descomponer el código cliente existente.

⌦ Puede ser que el código se complique más de lo que debería, ya que se introducen muchas nuevas interfaces y clases junto al patrón.

Singleton

Propósito

Nos asegura que una clase tenga una única instancia, proporcionando un punto de acceso global a dicha instancia.

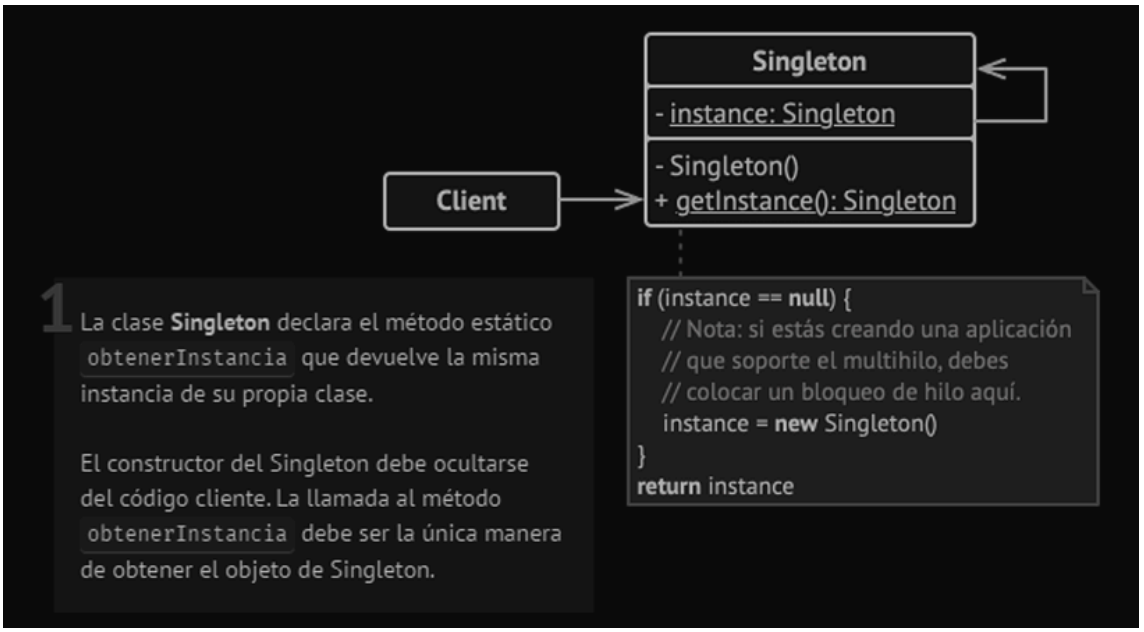
Problema / Solución

Resuelve 2 problemas, vulnerando el principio de responsabilidad única.

1. Garantiza que la clase tenga una única instancia.
2. Proporcionar un acceso global a dicha instancia.

Para esto, hacen privado el constructor por defecto (para evitar que se use el new),y luego crean un método de creación estático que actúa como constructor. Este método crea el objeto en caso de no existir y sino, devuelve la referencia a si mismo que la guarda estáticamente.

Estructura



Aplicabilidad

Sirve para tener un control más estricto de las variables globales.

Pros / Contras

- ✔ Una clase tiene una única instancia
- ✔ Tenes un punto de acceso global a dicha instancia.
- ✔ Solo se inicia cuando se lo requiere por primera vez.
- ⌦ Vulnera la responsabilidad única.
- ⌦ Puede enmascarar un mal diseño.
- ⌦ Requiere tratamiento especial para entornos con múltiples hilos de ejecución.
- ⌦ Es complicado hacerle pruebas unitarias en un framework por la naturaleza de su estructura.

Comparativa entre los 3 patrones.

Patrones	Objetivo	Problemas que resuelve	Solución que aporta.	Pros	Contras
Factory Method	Permite crear una interfaz común para crear objetos en una superclase, donde las subclases alteran el tipo de objetos que crean.	Cuando la aplicación escala y tenés que integrar múltiples opciones u objetos, se hace difícil.	Poder crear objetos utilizando el método de una interfaz que actúe de fábrica, devolviendo esta interfaz una clase abstracta. De dicha interfaz se cuelgan las fábricas que producirán estos objetos, y de dichas clases abstractas se cuelgan los objetos que producen estas fábricas.	Aumenta la flexibilidad para incorporar objetos. Evita el acoplamiento. Sigue el principio de responsabilidad única.	Es complicado de aplicar, por todas las subclases que requiere.
Abstract Factory	Permite crear una interfaz común para crear variantes de familias de objetos.	Es como el Factory, pero más magnificado, se centra más en familias de objetos, las cuales crean en si el mismo tipo de objeto, pero con sus propias variantes.	Incorporar una interfaz común, para crear objetos, los se crean mediante su propia fábrica. La interfaz devuelve una clase abstracta del producto, lo que permite luego devolver una variante concreta de una familia concreta con las subclases que implementen la interfaz común (la factoría abstracta).	Aumenta la flexibilidad para incorporar objetos. Evita el acoplamiento. Sigue el principio de responsabilidad única.	Es complicado de implementar, como el anterior.
Singleton	Asegura que una clase se instancie únicamente, dando un punto de acceso global a la misma.	Cuando necesitas controlar el acceso a una clase, o la cantidad de instancias de la misma.	Implementar una clase con un constructor privado, con un método para crear el objeto de sí misma. Si el objeto existe, devuelve una referencia de dicho objeto, y si no, la crea.	Más eficiente. Un punto de acceso global y una única instancia asegurados. La clase solo se crea cuando se la necesita.	Vulnera el principio de responsabilidad única. Puede enmascarar un mal diseño. Requiere tratamiento especial para multi-thread. Complica las pruebas unitarias en frameworks.

Patrones de diseño Estructurales

Adapter - wrapper

Propósito

Patrón de diseño enfocado en la colaboración entre objetos con interfaces incompatibles.

Problema – Solución

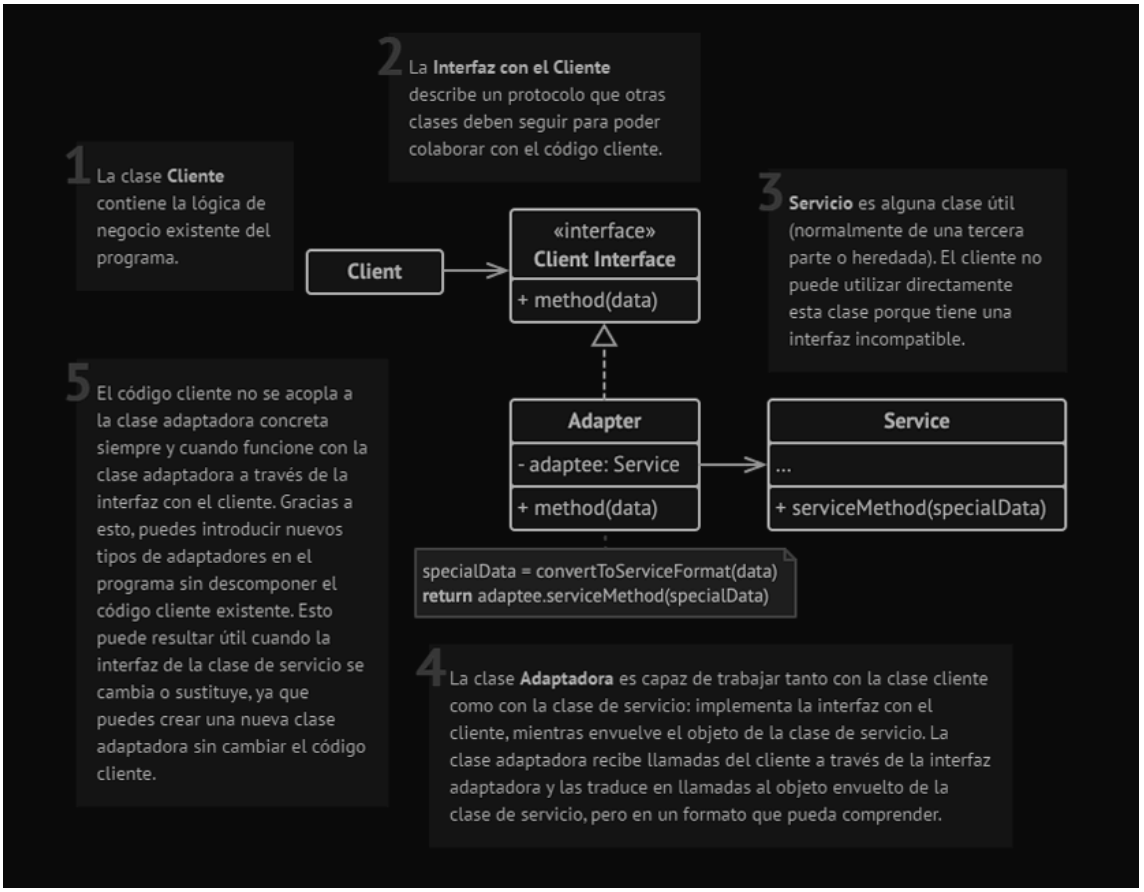
Tenés objetos cuyas interfaces son incompatibles, por ejemplo, te llegan los datos en XML, pero la biblioteca que los procesa necesita JSON.

La solución que propone este patrón, es implementar una clase adaptadora, que convierte la interfaz de un objeto para que otro objeto pueda entenderla.

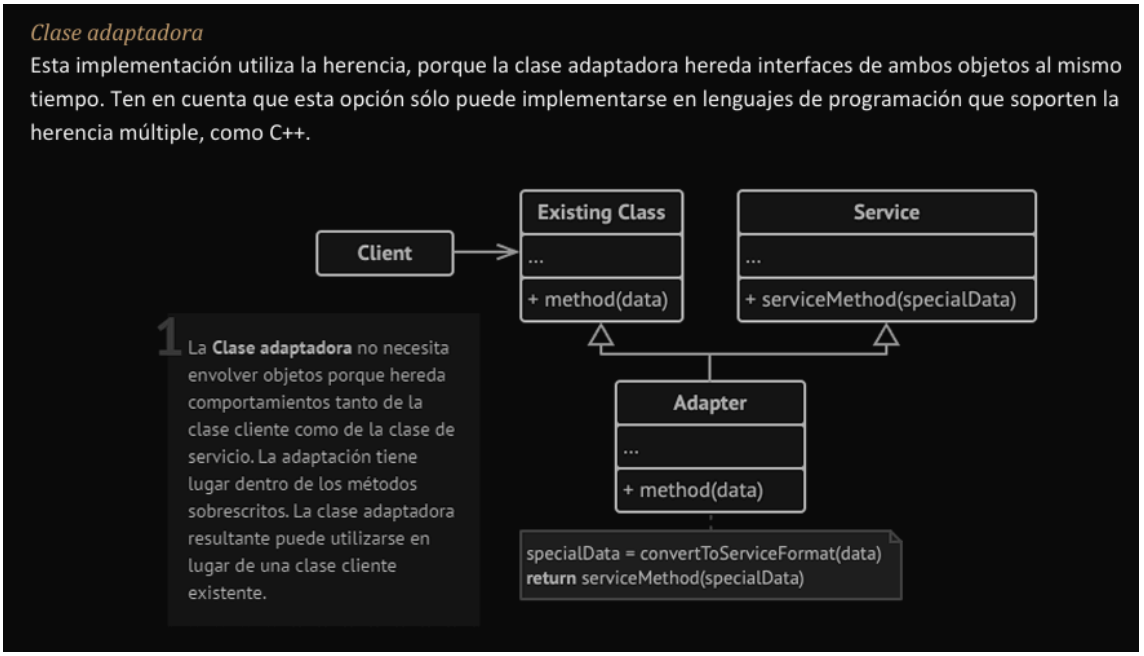
El adaptador obtiene una interfaz compatible con uno de los objetos existentes, luego mediante esa interfaz, el objeto puede invocar con seguridad los métodos del adaptador.

Cuando el adaptador recibe la llamada, el adaptador pasa esa solicitud al segundo objeto, previamente adaptándola en el formato y orden que el segundo objeto espera.

Estructura



La interfaz cliente es la que deben utilizar las otras clases para utilizar el adaptador.



Opción 2 totalmente terrorista.

Pros – Contras

- ✔ Sigue el principio de responsabilidad única.
- ✔ “Principio abierto/cerrado”: Te permite hacer nuevos adaptadores o adaptar más servicios sin descomponer el código cliente.
- ⌦ Aumento de complejidad del código, por la introducción de nuevas clases e interfaces.

Composite

Propósito

Componer objetos en estructuras de árbol y trabajar con dichas estructuras como si fueran objetos individuales. Solamente aplicable si se puede representar el modelo de la aplicación como un árbol.

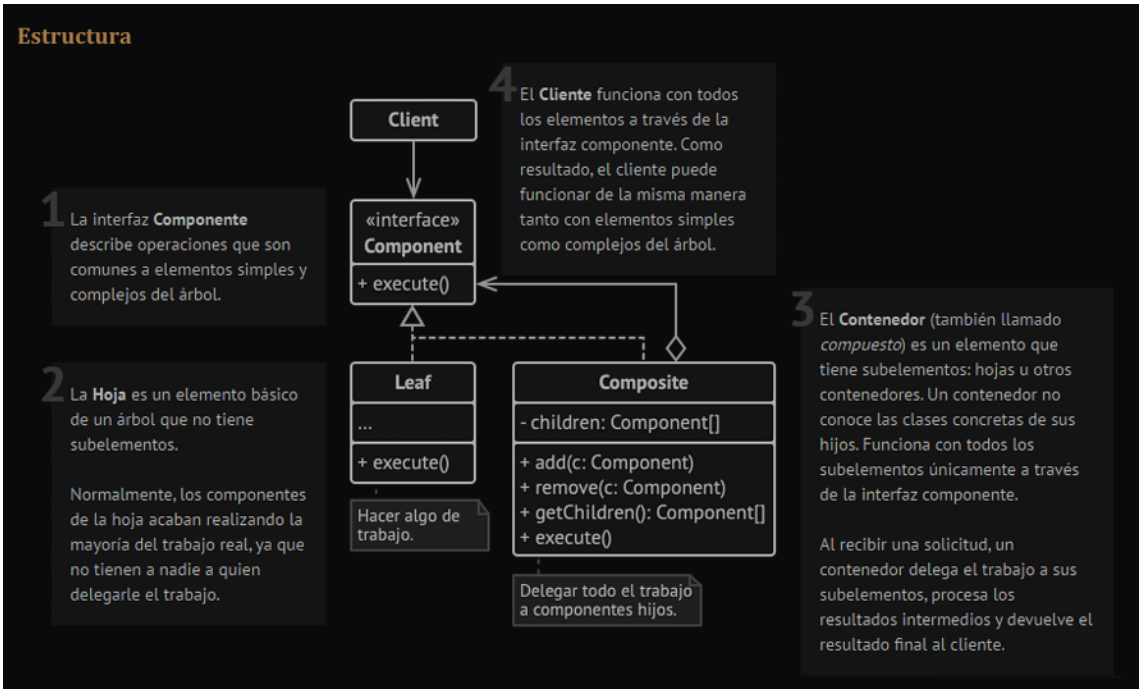
Problema – Solución

Tenés que manejar clases que dentro contienen otras clases, como una aplicación de pedidos.

Composite sugiere que para manejar esto, trates todo como cajas dentro de cajas, y descomponiendo cada caja llegas a un objeto base que es lo más básico que puede contener un pedido.

La idea es trabajar con productos y cajas a través de una interfaz común.

Estructura



El cliente únicamente interactúa con la interfaz “Componente”, la cual contiene la estructura de árbol mencionada. No conoce la lógica detrás, pero si es un objeto básico se comportará de una forma y si es una combinación de productos se comportará de otra.

Pros – Contras

Úsalo solo cuando tengas que implementar una estructura de objetos con forma de árbol, además de querer que el cliente trate los elementos simples y complejos por igual.

- ✔ Usas el polimorfismo y recursión a fondo.
- ✔ “Principio de abierto / cerrado”
- ⌦ Si las clases difieren demasiado en funcionalidad, proveer una interfaz común es complicado.

Facade

Propósito

Proporcionar una interfaz simplificada a una biblioteca, framework o un grupo complejo de clases. Enmascarar la complejidad básicamente.

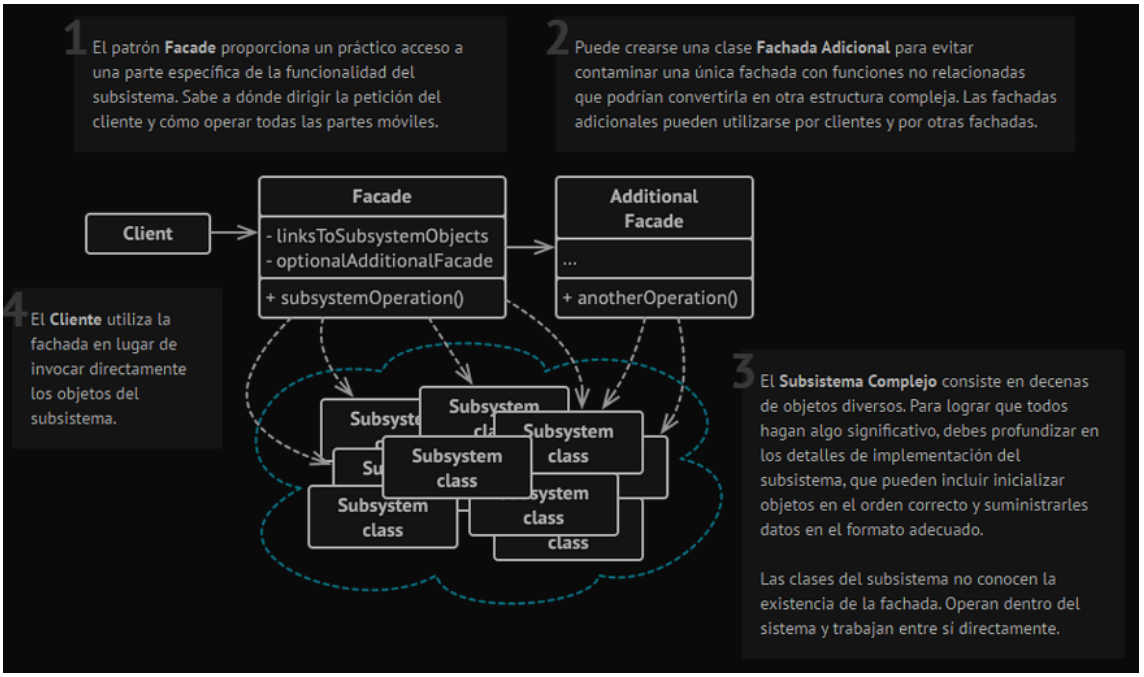
Problema – Solución

Tener un código que trabaja con muchos objetos que pertenecen a una biblioteca o framework, donde tenes que inicializar cada objeto, registrar dependencias y demás. Por lo que TU código, el que maneja la lógica de negocio, se ve acoplado a los detalles de la implementación de las clases de otros, haciéndolo más difícil de entender y mantener.

Facade te propone crear una clase que provea una interfaz simple, donde por detrás haya un subsistema complejo que contiene varias partes móviles. Por ejemplo, si en Qt utilizas 2 o 3 boludeces, hacer una

clase intermedia que se encargue de inicializarlas y demás, y vos uses esos componentes de Qt a través de tu clase “fachada” que los maneje, en vez de implementarlos vos mismo en cada clase que haces.

Estructura



Pros – CONTRAS

- ✔ Útil para un subsistema por capas.
- ✔ Puede aislar el código de la complejidad del subsistema.
- ⌦ Puede convertirse en un objeto todopoderoso acoplado a todas las clases de una aplicación.

COMPARATIVA

Patrón	Objetivo	Problema	Solución	Pros	Contras
Adapter	Permitir la colaboración entre objetos con interfaces incompatibles.	Objetos que funcionan de manera distinta, ya sea por la interfaz o por la forma en la que necesitan obtener los datos.	Sugiere crear una clase “Adaptadora” y una interfaz para el cliente. Cliente debe comunicarse con el adaptador a través de dicha interfaz, la cual le permite comunicarse con el objeto con seguridad. El Adaptador implementa dicha interfaz y contiene la referencia al servicio que adapta. Al recibir la petición, la adapta al formato que use el servicio correspondiente y envía la solicitud a dicho servicio.	Principio abierto/cerrado, se pueden incluir más adaptadores y adaptar más servicios. Se desacopla del código cliente. Principio de responsabilidad única.	Puede ser complicado de implementar por las clases e interfaces que introduce.
Composite	Permitir que un cliente interactúe con elementos básicos o complejos por igual.	Múltiples objetos compuestos, donde a su vez pueden estar dentro de otros objetos.	Crear una interfaz común “Componente” la cual use el cliente para interactuar con los objetos, dicha interfaz es implementada por clases **distinguidas como hoja o intermedias**, donde las clases hojas serían las clases que no pueden albergar otros objetos, y serían lo más básico de la arquitectura. Las clases intermedias o composiciones son clases que pueden contener a otras, y lo hacen teniendo una composición de la propia interfaz, de tal forma que pueden albergar tanto objetos básicos como complejos. Dichas clases intermedias albergan métodos para sumar hijos, quitarlos, consultarlos, y ejecutar lo que tengan que hacer.	Principio de abierto/cerrado Poder usar el polimorfismo y la composición. // nota de yo, se sabe que el polimorfismo empeora el rendimiento del código por lo que realmente no sería un pro, pero si lo re simplifica para programarlo xd	Solo se puede aplicar en sistemas que se puedan representar como un árbol. Si la complejidad de los objetos aumenta, resulta complejo generalizar todo con una sola interfaz.
Facade	Enmascarar la lógica compleja de distintos subsistemas mediante una única interfaz con la cual el cliente se comunique.	Utilizar grandes bibliotecas o frameworks, los cuales requieran un seguimiento estricto, o la dependencia del código cliente de varios sistemas complejos volviéndose la lógica del negocio dependiente de programas de terceros.	Crear una clase “Fachada” que se encargue de manejar estos subsistemas, con una interfaz clara que pueda usar el cliente para hacer lo que necesite.	Principio de abierto/cerrado Oculta la lógica al cliente. Desacopla el código cliente de dependencias de terceros.	Puede volverse una clase todopoderosa que maneje muchísimas dependencias.

Patrones de comportamiento

Observer – Listener

Propósito

Permite definir un mecanismo de subscripción para notificar a varios objetos sobre cualquier evento que le suceda al objeto que están observando.

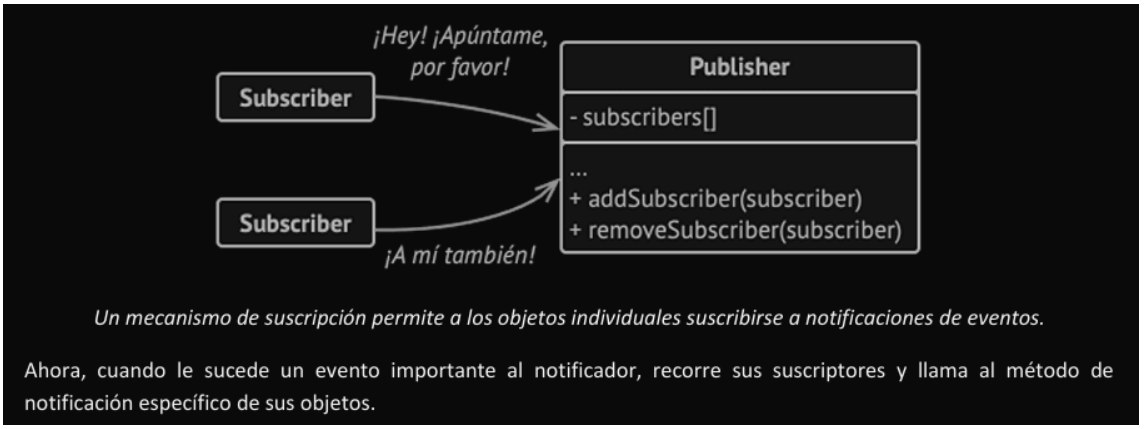
Problema – solución

Querés manejar un esquema de subscripciones para saber, por ejemplo, el estado de un envío?

Este esquema lo resuelve utilizando un objeto llamado notificador o publicador. Los objetos que quieran conocer el estado del notificador, se les llaman suscriptores.

Sugiere que añadas un mecanismo de suscripción a la clase notificadora para que los objetos individuales puedan suscribirse o cancelar la suscripción a un flujo de eventos provenientes de la clase notificadora.

Para esto usas un campo matriz que almacene la lista de referencias a objetos subscriptores, y métodos que permitan añadir o quitar suscriptores de la lista.



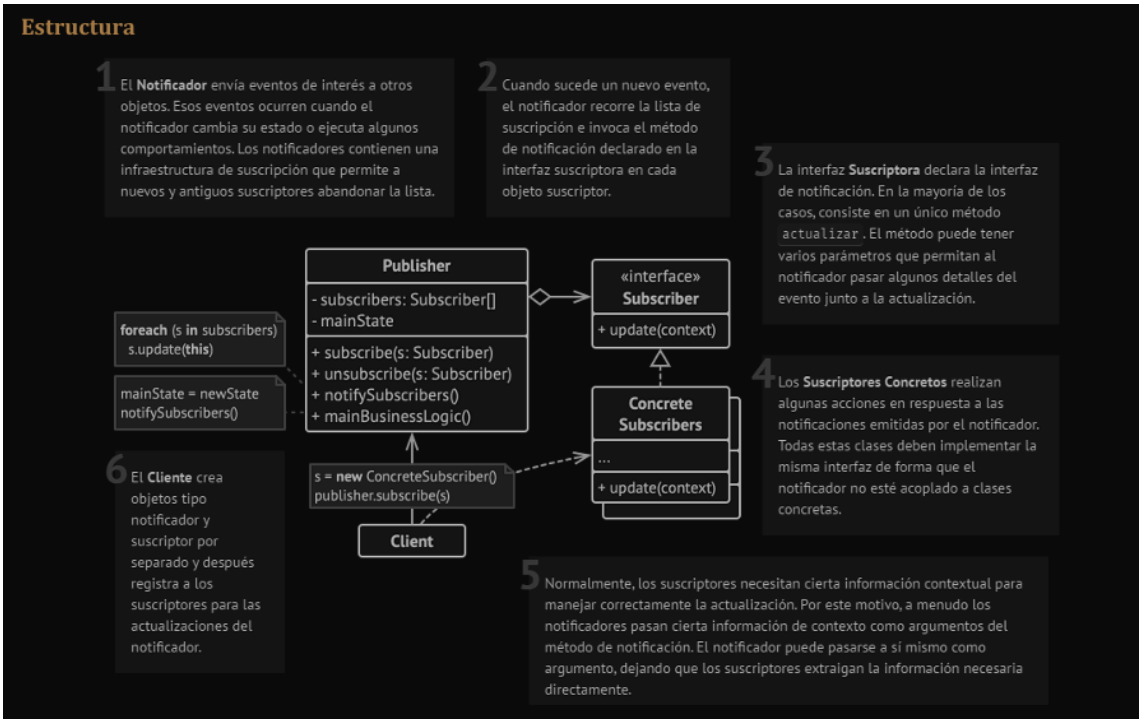
Un mecanismo de suscripción permite a los objetos individuales suscribirse a notificaciones de eventos.

Ahora, cuando le sucede un evento importante al notificador, recorre sus suscriptores y llama al método de notificación específico de sus objetos.

Para que esto suceda, es FUNDAMENTAL que los suscriptores implementen la misma interfaz, y que el notificador únicamente se comunique con ellos mediante esta interfaz (lo van a ver en la estructura).

La interfaz debe declarar un método de notificación y unos parámetros necesarios para que le notificador pueda pasar información contextual de la notificación en caso de ser necesario.

Estructura



Aplicabilidad

Hay que usarlo cuando el estado de un objeto pueda necesitar cambiar el de otros objetos, y el grupo de dichos objetos sea desconocido de antemano o cambie dinámicamente.

Pros / contras

- ✔ Introducir nuevas clases suscriptoras es easy.
- ✔ Permite establecer relaciones entre objetos durante el tiempo de ejecución.
- ⌦ Los suscriptores son notificados en un orden aleatorio.

Strategy

También llamado: Estrategia
??

Propósito

Definir una familia de algoritmos, colocarlos en clases separadas y hacer sus objetos intercambiables.

Problema – fix

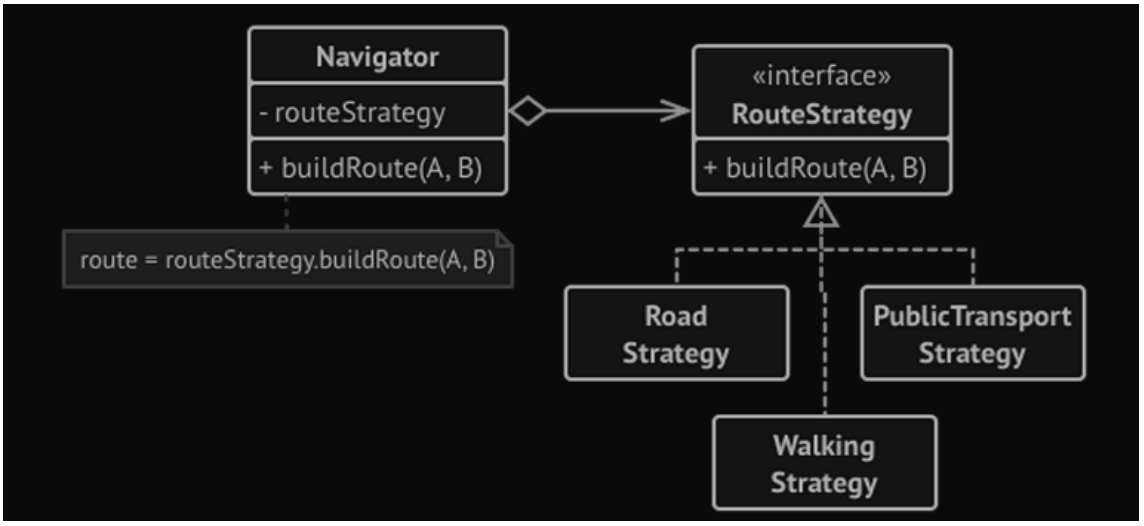
Un GPS; tenes múltiples formas de calcular la misma ruta, sea por agua, a pie, caminando, volando, tpeandote. Armar el algoritmo para adaptar la forma de calcular la ruta es un quilombo.

Si tenes una clase que hace esos algoritmos distintos, Strategy sugiere que la agarres y separes esos algoritmos en distintas clases llamadas **estrategias**.

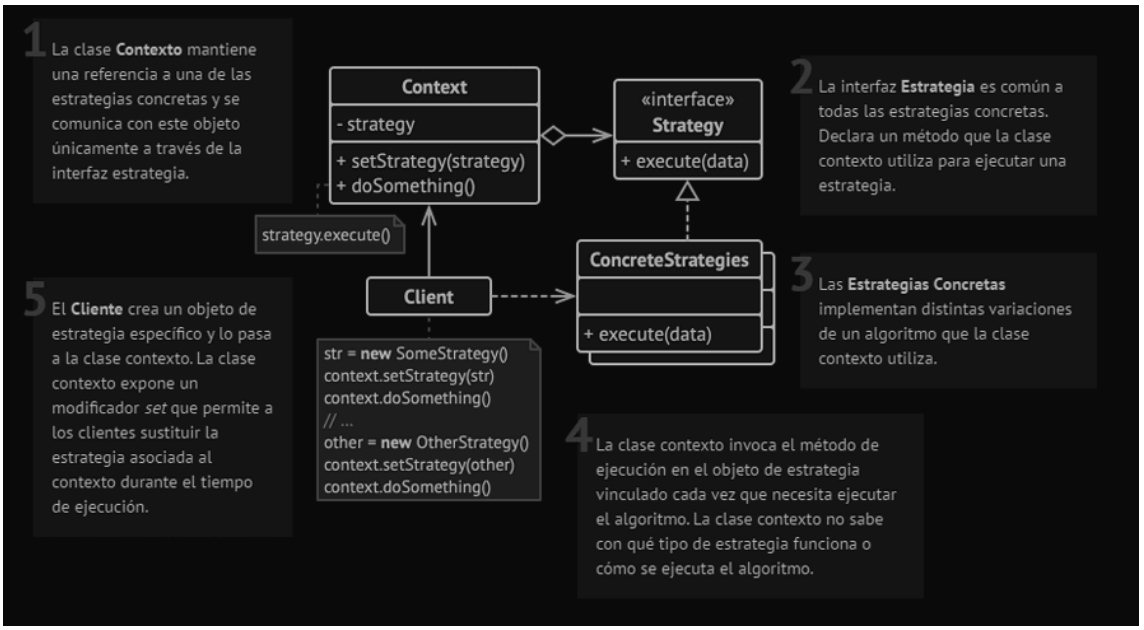
La clase original, que se le dice **contexto**, necesita tener un campo para almacenar una referencia a una estrategia. Luego, el contexto delega el trabajo a el objeto estrategia que tenga vinculado, en vez de ejecutarlo por su cuenta.

La clase contexto no es quien elige la estrategia a ejecutar, sino el cliente. Todo funciona a través de la misma interfaz genérica, por lo que solo expone un único método para disparar el algoritmo encapsulado para la estrategia seleccionada. (ósea tiene 2 métodos).

Esto es para que el contexto se vuelva independiente de las estrategias concretas, para poder añadir nuevos algoritmos o modificar los existentes fácilmente.



Estructura



Aplicabilidad

Aparte de usarlo cuando tenes una clase que ejecuta múltiples algoritmos, pero solo 1 a la vez, podes usarlo cuando tenes muchas clases similares que sólo se diferencian en la forma en la que ejecutan cierto comportamiento.

También sirve para aislar la lógica del negocio de una clase, de los detalles de implementación de algoritmos que no pueden no ser importantes en el contexto de esa lógica.

Y, sobre todo, si tu clase tiene un enorme operador condicional que hace variar el algoritmo.

Pros / CONTRAS

- ✔ Poder intercambiar algoritmos usados dentro de un objeto durante ejecución.
- ✔ Poder aislar detalles de implementación de un algoritmo del código que lo usa.
- ✔ Poder sustituir herencia por composición (menos acoplamiento).
- ⌦ No se debe aplicar si solo son un par de algoritmos que raramente cambian.
- ⌦ Los clientes deben saber las diferencias entre estrategias para seleccionar la adecuada.
- ⌦ Muchos lenguajes permiten soporte tipo funcional para implementar distintas versiones de un algoritmo en un grupo de funciones anónimas, haciendo que no sea necesario inundar de clases e interfaces el código.

Comparativa

Patrones	Objetivo	Problema	Solución	Pros	Contras
Observer	Estar pendiente de un flujo o del estado de otro objeto, para notificar a otros objetos.	Tener que estar programando formas de que un objeto se entere que el estado de otro cambió.	Implementar una clase Publisher o notificadora, que contenga una lista de suscriptores y métodos para poder suscribirse/darse de baja, junto a métodos para notificar a los objetos suscriptores. Se comunica mediante una interfaz suscriptora con los objetos suscriptores, la cual dichos objetos deben implementar para poder suscribirse.	✔ Introducir clases suscriptoras es fácil. ✔ Poder cambiar las relaciones entre objetos en tiempo de ejecución.	⌦ Los suscriptores son notificados en un orden aleatorio.
Strategy	Poder cambiar los algoritmos que se usan para solucionar un problema en tiempo de ejecución.	Tener que planificar de distintas maneras la solución a un problema, o tener una clase con muchos algoritmos juntos donde solo se ejecutan uno solo a la vez.	Implementar una clase Contexto, que tenga un campo para referenciar estrategias, junto a métodos para seleccionarlas y dispararlas. Estas estrategias no son más que clases o interfaces que selecciona el cliente para poder resolver el problema. El contexto no tiene conocimiento de las estrategias, ni las elige.	✔ Menos acoplamiento. ✔ Poder intercambiar algoritmos en tiempo de ejecución. ✔ Aislar detalles de la implementación de un algoritmo que un código usa. ✔ Es fácil agregar nuevas estrategias.	⌦ No se debe aplicar si solo hay un par de algoritmos que rara vez cambian. ⌦ Los clientes deben conocer las estrategias. ⌦ En lenguajes modernos es posible reemplazar el Strategy por un grupo de

					funciones anónimas.

Unidad 5: Calidad de software

Qué es?

Es la **medida** en la que un producto de software cumple con los requisitos y expectativas del cliente, así como los estándares y prácticas de desarrollo establecidos. Trata de la capacidad de cumplir su propósito de manera efectiva, eficiente y libre de errores, satisfaciendo las necesidades de los usuarios y las partes interesadas.

Los **atributos de la calidad de software** son factores que contribuyen a la calidad general del producto de software.

Importancia en la industria

1. Satisfacción del cliente.
2. Reducción de costos.
3. Eficiencia y productividad.
4. Competitividad.

Consecuencias de la baja calidad de software.

1. Errores y defectos.
2. Frustración del Usuario.
3. Retrasos en el desarrollo.
4. Costos Adicionales.
5. Impacto en la Reputación.
6. Pérdida de Clientes.
7. Seguridad en Riesgo.
8. Dificultad en Mantenimiento.
9. Dificultad en Escalabilidad.
10. Incumplimiento normativo.

Atributos de la calidad de software

1. **Funcionalidad**: La capacidad del software de cumplir las tareas y funciones especificadas de manera correcta y completa. Implica que el software cumple con todos los requisitos funcionales definidos sin presentar comportamientos raros o errores.
2. **Fiabilidad**: Capacidad de funcionar de manera consistente y libre de fallos por un tiempo determinado.
3. **Usabilidad**: Facilidad de uso y experiencia de usuario.
4. **Eficiencia**: Capacidad de realizar las funciones de manera rápida y con uso eficiente de recursos del sistema.
5. **Mantenibilidad:** Capacidad del software de modificarse, actualizarse y mantenerse a largo plazo.
6. **Portabilidad**: Capacidad de funcionar en distintas plataformas y entornos sin cambios significativos.
7. **Seguridad.**
8. **Compatibilidad**: Capacidad del software para funcionar correctamente junto a otros sistemas.
9. **Cumplimiento normativo.**
10. **Escalabilidad.**
11. **Documentación**: No siempre mencionado pero importante.

Metodologías y estándares de Calidad

ISO 9001

Es el estándar internacional de gestión de calidad que aplica a una amplia gama de industrias. Establece un enfoque sistemático para gestionar la calidad de todos los aspectos de un proyecto de software, desde la planificación hasta la entrega y soporte.

Capability Maturity Model Integration (CMMI).

Conjunto de prácticas para mejorar los procesos del desarrollo de software. Da un marco de referencia para medir y mejorar la madurez de procesos de una organización.

Ayuda a las organizaciones a evaluar y mejorar sus procesos.

Agile (Scrum, kanban, etc)

xd

ITIL (Information Technology Infastructure Library)

Conjunto de prácticas recomendadas para gestión de servicios de TI.

IEEE 730

Norma que establece pautas para gestionar la calidad del software. Describe procesos de aseguramiento y control de calidad.

Six Sigma

Se enfoca en reducir defectos y mejorar la calidad a través de la medición y análisis de datos.

DevOps

Es una cultura y conjunto de prácticas que fomentan la colaboración entre los equipos de desarrollo y operaciones. Caracterizado por automatización de pruebas, despliegue continuo y monitoreo.

Mejoran la calidad reduciendo errores de entrega, acelerar el tiempo de comercialización y garantiza mejor colaboración entre equipos.

Técnicas para mejorar la calidad del software

1. **Pruebas de software.**
2. **Revisión de código.**
3. **Gestión de configuración**: Herramientas de control de versiones o de gestión de dependencias.
4. **Mantenibilidad del Código.**
5. **Pruebas de Rendimiento y Seguridad.**
6. **Documentación**: Documentar el código y tener documentación para los usuarios.
7. **Estándares de Codificación.**
8. **Gestión de Proyectos y Comunicación.**
9. **Desarrollo Sostenible**: Utilizar diseño modular, principios SOLID.
10. **Prácticas de DevOps.**
11. **Usar IA:** aguante codex y github copilot wacho.

Mejoras de proceso

No hace falta definir mucho esto, pero lo más importante viene acá, como definir los procesos.

1. Analizo la situación actual, identifico que procesos que se llevan a cabo hoy en día. (AS-IS)
2. Establezco objetivos de mejora. (TO-BE)
3. Comprendo la organización.
4. Diseño un plan de proyecto.
5. Implemento el proyecto.
6. Monitoreo y evalúo los resultados, comparando el desempeño actual con los objetivos planteados.

Tipos o enfoques específicos dentro de la mejora de procesos.

Mejora de procesos (PI = Process Improvement)

Es una definición general, se refiere a cualquier esfuerzo sistemático para mejorar la eficacia, eficiencia, calidad o adaptabilidad de un proceso existente. Se puede aplicar a cualquier tipo.

Mejora de Procesos de Negocio (BPI)

Subconjunto específico enfocado a procesos de negocio.

El objetivo es hacer que los procesos centrales de una organización sean más eficientes y alineados con los objetivos estratégicos.

Se suele usar el Modelado de Procesos, reingeniería, Six Sigma y Lean.

Mejora de Procesos de Software (SPI)

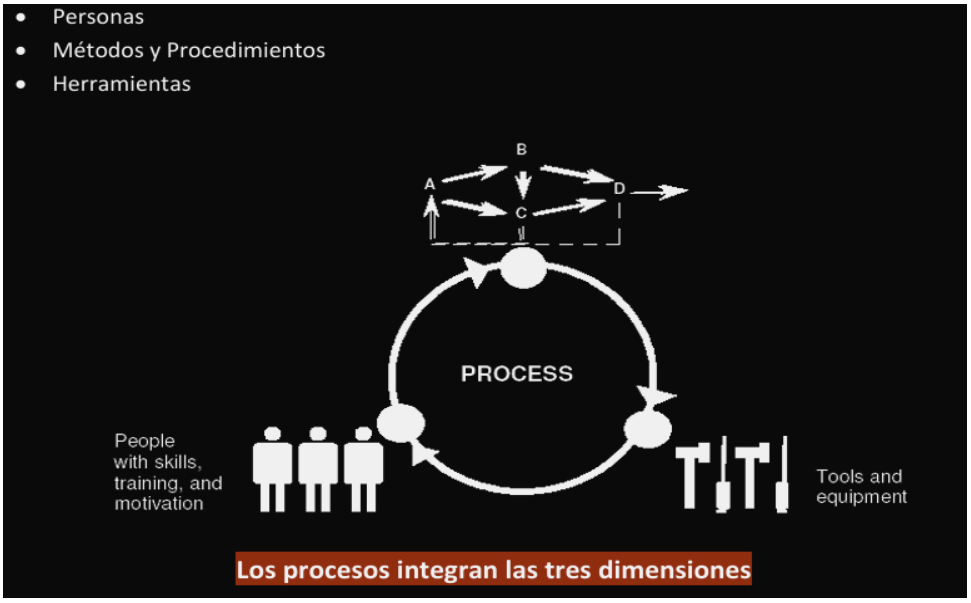
Mejora sistemática de procesos utilizados para desarrollar y mantener software.

Técnicas y marcos comunes: CMMI, ISO/IEC 12207 y 15504 SPICE, DevOps, Agile, Scrum.

Diferencias

Tipo	Ámbito	Ejemplo principal
Process Improvement (PI)	General	Mejorar el flujo de trabajo de atención al cliente
Business Process Improvement (BPI)	Negocios y gestión organizacional	Rediseñar el proceso de facturación
Software Process Improvement (SPI)	Desarrollo de software	Implementar integración continua (CI)

Elementos relevantes en un proyecto de PI



Personas

Son quienes ejecutan, supervisan y mejoran los procesos.

- ✔ Sin personas motivadas y capacitadas, no funciona el proceso.
- ✔ Involucrar actores clave permite identificar problemas reales y soluciones viables.
- ✔ Una gestión del cambio adecuada ayuda reducir la resistencia.

Métodos y Procedimientos

Son la forma estructurada en la que se realizan las actividades.

- ✔ Permiten estandarizar el trabajo, reduciendo errores y variaciones.
- ✔ Ayudan a medir, controlar y mejorar continuamente actividades.
- ✔ Facilitan la transferencia de conocimiento y la capacitación de nuevos empleados.

Herramientas

Soportan y facilitan la ejecución, monitoreo y mejora de los procesos. Pueden ser software de BPM; diagramas de flujo, hojas de cálculo, automatización, etc.

- ✔ Permite visualizar, medir y controlar procesos.
- ✔ Aumenta la eficiencia al reducir tareas manuales o repetitivas.
- ✔ Facilitan la toma de decisiones basada en datos.

Proceso de Desarrollo de Software

1. Recolección de requisitos

• Entrevistas con el cliente y usuarios

• Análisis de necesidades funcionales y no funcionales

• Redacción del documento de requerimientos

2. Análisis y diseño

• Diseño arquitectónico del sistema

• Modelado de base de datos, interfaces y lógica del sistema

• Selección de tecnologías

3. Desarrollo

• Codificación del software según los requerimientos

• Control de versiones y pruebas unitarias

4. Pruebas

• Pruebas funcionales, de integración, de rendimiento y de usuario (UAT)

• Corrección de errores detectados

5. Implementación y despliegue

• Instalación en ambiente productivo

• Capacitación a usuarios finales

• Migración de datos (si aplica)

6. Mantenimiento

• Soporte post-implementación

• Actualizaciones y mejoras

• Corrección de errores no detectados inicialmente

Proyecto de software vs Proceso de mejora

Actividades	Mejora de Procesos	Desarrollo de Software
Análisis inicial	Diagnóstico del proceso	Recolección de requisitos
Diseño	Rediseño del proceso (TO-BE)	Análisis y diseño del sistema
Ejecución	Implementación de mejoras	Codificación y pruebas
Validación y ajuste	Seguimiento y evaluación	Testeo, despliegue, mantenimiento

Las actividades que componen un proyecto de software son distintos de las que componen un proceso de mejora de actividades. Ambos proyectos son estructurados y orientados a la mejora o creación de soluciones.

Un proyecto de mejora de procesos puede requerir desarrollar un software como parte de la solución, mientras que un proyecto de software puede **iniciar con la mejora de procesos**, no automatizar ineficiencias.

Aspecto	Mejora de Procesos	Desarrollo de Software
Enfoque principal	Procesos (personas, tareas, flujos)	Tecnología (software, código)
Resultado esperado	Procesos optimizados	Producto software funcional
Cambios típicos	Procedimientos, roles, herramientas	Código, interfaces, bases de datos
Requiere programación	No necesariamente	Sí, es esencial

Definir objetivos

Mejorar tiene un costo, y hacerlo genera un impacto directo en el negocio, por lo que identificar los objetivos permite evaluar el éxito de la mejora una vez implementada.

Planificar el proyecto

Se usan técnicas de administración de procesos. La idea de planificarlo es lograr el objetivo en el plazo determinado con el presupuesto dado. Los esfuerzos de mejora tienen que ser administrados como proyectos. Acá se gestionan tareas, personas, se requieren recursos y se deben gestionar adecuadamente los riesgos.

Armar equipos

1. Para distribuir responsabilidades.
2. Aprovechar distintas habilidades.
3. Facilitar toma de decisiones.
4. Aumentar el compromiso.
5. Mejorar la comunicación.

Roles Principales

Los roles estándar suelen ser:

- Líder o jefe de proyecto: Coordina al equipo, planifica las actividades, gestiona tiempos y recursos.
- Analista de Procesos/ Negocio: Releva, documenta y analiza los procesos actuales.
- Especialista Técnico / Desarrollador.
- Usuario Clave (Key User): Representa a los usuarios del proceso o sistema.
- Responsables de Calidad o Mejora Continua.
- Sponsor o Patrocinador

Modelo CMMI

Este modelo se basa en un “Modelo de Madurez de Capacidades”, la cual es una estructura que define como una organización puede mejorar progresivamente sus procesos para lograr mejores resultados, ser más consistentes, eficientes y predecibles.

Ahora estudiamos el CMMI, Capability Maturity Model Integration, desarrollado por el SEI.

Uso

- Medir nivel de madurez de una organización.

- Guiar a mejora continua de procesos.
- Aumentar la calidad del producto o servicio.
- Reducir riesgos en proyectos complejos.

Niveles de madurez en CMMI

Nivel	Nombre	Descripción breve
1	Inicial	Procesos impredecibles, reactivos. El éxito depende de personas, no de procesos.
2	Gestionado	Los proyectos se planifican, ejecutan y controlan. Se siguen prácticas básicas.
3	Definido	Los procesos están documentados, estandarizados y mejorados en toda la organización.
4	Cuantitativamente gestionado	Se usan métricas para gestionar procesos y controlar la calidad.
5	En optimización continua	Se mejora continuamente a través de innovaciones y análisis de desempeño.

Ejemplos de cada nivel:

1. Nivel 1 (Inicial): No hay procesos definidos, cada programador trabaja a su manera. A veces entregan a tiempo, otras no. Los errores se detectan tarde y cuesta caro.
   Por lo tanto el **éxito** depende de las personas individuales, no de un sistema o proceso.
2. Nivel 2 Gestionado: Se implementan herramientas de gestión de proyectos, cada proyecto tiene un plan y responsables definidos, se registran problemas y se realizan reuniones de seguimiento.
   Por lo tanto, ya no dependen tanto de la experiencia individual, y los proyectos se empiezan a cumplir con más regularidad.
3. Nivel 3 Definido: Establecen un proceso estándar para todos los equipos.
   Documentan como inicia el proyecto, como probar el software, y como liberar versiones, además de proveer capacitaciones.
   Por lo que, los equipos trabajan de forma más coordinada, con menos errores y más eficiencia.
4. Nivel 4 Cuantitativamente gestionado: Se comienzan a medir tiempos reales de desarrollo, cantidad de errores por versión y el rendimiento de cada proceso. Se usan dichos datos para predecir riesgos y mejorar estimaciones.
5. En optimización continua: Se analizan constantemente los datos para buscar formas de mejorar los procesos, y se promueve la innovación interna y mejora continua.
   Por lo tanto la empresa se vuelve altamente competitiva, con entregas confiables y productos de alta calidad.

Beneficios de la Certificación CMMI

1. Ser mejor proveedor.
2. Clientes más satisfechos y fieles.
3. Mayor rentabilidad.
4. Mejor ambiente laboral
5. Competitividad.
6. Incremento en las ventanas.

Armado de equipos

- Los equipos se tienen que alinear con los procesos definidos.

- Deben existir roles claros y responsables designados.
- La comunicación entre áreas debe ser fluida.
- Se fomenta la mejora continua desde todos los niveles.

Equipos claves en una organización que implementa CMMI

1. **Equipo de mejora de procesos**:
   Se encarga de definir, mantener y mejorar los procesos.
   Suele incluir miembros de diversas áreas: desarrollo, calidad, gestión, etc.
   Rol fundamental en niveles 3, 4 y 5.
2. **Equipos de proyectos**
   Equipos operativos que ejecutan proyectos siguiendo los procesos definidos.
   Tienen un líder de proyecto (Project Manager) y miembros técnicos.
   En niveles 2 y superiores, deben usar herramientas de seguimiento, gestión de riesgos, control de calidad, etc.
3. **Grupo de aseguramiento de calidad (SQA)**
   Verifica que los procesos se estén cumpliendo correctamente.
   Auditan proyectos y procesos internos.
   En niveles 2 en adelante, este grupo se vuelve esencial.
4. **Equipo de métricas o análisis**
   En niveles 4 y 5, se requiere gestión cuantitativa.
   Este equipo recopila y analiza datos sobre calidad, tiempo, defectos, productividad, etc.
   Apoya la toma de decisiones basadas en evidencia.
5. **Alta dirección y líderes funcionales.**
   No son “equipos” operativos, pero su compromiso es clave.
   Proveen recursos, guía y autoridad para implementar mejoras.

Diagramas para representar modelos de madurez

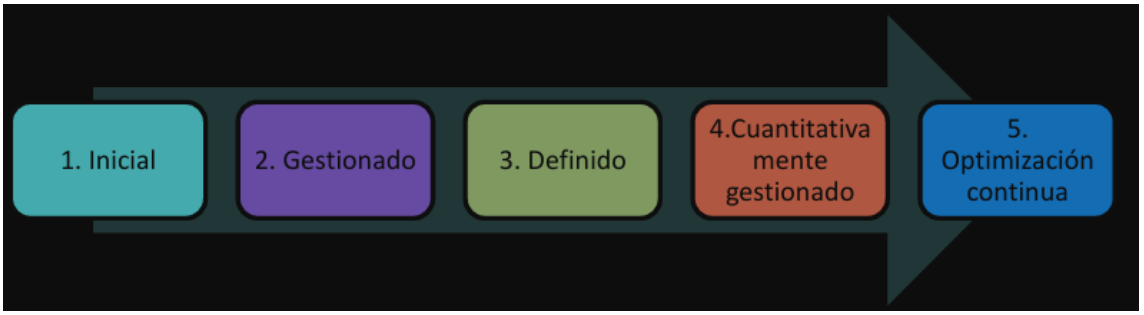
Pirámide de niveles de madurez

Representa los 5 niveles como escalones de una pirámide o escalera, mostrando que cada nivel se construye sobre el anterior.



Diagrama de progreso lineal

Un diagrama de tipo timeline horizontal donde se indican los 5 niveles de madurez como etapas consecutivas, útil para mostrar acciones o indicadores clave por nivel:



Debajo de cada nivel se agrega, prácticas clave, indicadores, riesgos o beneficios.

IA con el CMMI -> seguro no entra

Es relevante porque en el nivel más alto donde se busca una mejora continua, la IA facilita las cosas para proveerla.

La IA puede automatizar la mejora de procesos a través de análisis predictivos, aprendizaje automático y generación de recomendaciones.

Automatización Inteligente de tareas (Nivel 3 a 5)

eso

Gestión Cuantitativa y Predicción (Nivel 4)

La IA puede detectar patrones en grandes volúmenes de datos de proyectos, predecir riesgos o retrasos y recomendar mejoras.

Apoya en la toma de decisiones

Gracias a los análisis de la IA.

IA Como herramienta habilitadora de procesos

Mejora la eficiencia, consistencia y trazabilidad de actividades.

Especialista Técnico / Desarrollador

Unidad 6: Gestión de versiones

Definiciones previas

Gestión de cambios de software

Es el proceso de identificar, documentar, evaluar y controlar los cambios propuestos en un proyecto de software. Incluye evaluar como estos cambios afectarán al proyecto en términos de tiempo, recursos y riesgos.

Su objetivo es asegurar que cualquier cambio de requisitos, diseño o implementación se haga de manera eficiente, minimizando riegos y asegurando no comprometer objetivos del proyecto.

Control de cambios

Es un subconjunto dentro de la gestión de la configuración que se enfoca específicamente en cómo se aprueban, implementan y rastrean los cambios realizados en el software. El control de cambios establece un proceso formal para solicitar, evaluar, aprobar o rechazar cambios.

Su objetivo es garantizar que los cambios sean aprobados adecuadamente y no afecten negativamente el software ni los objetivos del proyecto.
Es un proceso formal.

Control de versiones

Es el proceso de gestionar múltiples versiones o variantes de un software a lo largo del tiempo. El control de versiones asegura que los desarrolladores puedan trabajar de manera colaborativa en diferentes características o correcciones, y que se pueda volver a versiones anteriores si es necesario.

La idea es proveer un historial de todos los cambios realizados en el código o en otros componentes, permitiendo la colaboración, la recuperación de versiones anteriores y la comparación de cambios.

Un ejemplo de esto es Git, que gestiona las versiones del código fuente. Es un proceso técnico.

Gestión de configuración de software (SCM)

Es el proceso de gestionar de manera sistemática todos los artefactos de un proyecto de software, como el código fuente, la documentación, los entornos de desarrollo y los componentes necesarios para la construcción de un producto. SCM asegura que todas las versiones y componentes estén bajo control y disponibles para el equipo.

Su objetivo es proporcionar control y trazabilidad sobre los elementos del sistema a lo largo del ciclo de vida del proyecto, asegurando que cada cambio se realice de forma estructurada y controlada.

Profundizando el conocimiento sobre control de versiones

En equipos de DevOps, es super útil porque puede reducir el tiempo de desarrollo y aumentar las implementaciones exitosas.

La idea del software de control de versiones es que realiza un seguimiento de las modificaciones del código en un tipo especial de base de datos, para que si se comete un error, se pueda volver hacia atrás fácilmente.

Constantemente en los equipos de software hay múltiples desarrolladores cambiando partes del código, por ejemplo uno puede estar fixeando un bug mientras otro está implementando una nueva feature.

El control de versiones permite el trabajo concurrente haciendo un seguimiento individual de cada colaborador, y evitando que entren en conflicto entre sí.

Importancia del control de versiones en proyectos de software.

Los sistemas de control de versiones (VCS) también les suelen llamar herramientas de SCM (Gestión del código fuente) o RCS (Sistema de control de revisiones). La más popular hoy en día es **Git**.

- ✔ Es la práctica recomendada para equipos de software y DevOps de alto rendimiento.
- ✔ Ayuda a las desarrolladoras a moverse más rápido.
- ✔ Mantiene la eficiencia y agilidad a medida que el equipo escala incluyendo más devs.

Ventajas nombradas por el texto:

- ✔ Historial completo de cambios a largo plazo para todos los archivos: Evitas tener el mismo archivo bajo distintos nombres para tenerlo divido por versiones.
- ✔ Creación de ramas y fusiones: Útil para mantener distintos flujos de trabajo independientes, para luego fusionarse.
- ✔ Trazabilidad: Ser capaz de seguir cada cambio es fundamental. Acá se mencionan otras herramientas como Jira.

Problemas que surgen al no usar control de versiones

1. Pérdida de código: Podes sobrescribir o eliminarse sin querer el código, ni hablar de si se meten varios locos a laburar en el mismo código.
2. Dificultad para colaborar en Equipo: Se cagan a palos para coordinar como incluir los cambios.
3. Falta de historial de Cambios: No te podés dar cuenta si fue mariano el que borró el dashboard de la página web.
4. Imposibilidad de revertir Cambios: No podés revertir metidas de pata o errores críticos, como cuando mariano borro la preruta de la página web.
5. Problemas de Lanzamientos de versiones: Complejiza el desplegar un software.
6. Complicaciones de resolución de conflictos.
7. Ausencia de Trazabilidad y Auditoria.
8. Falta de seguridad en el código.
9. Dificultad para implementar entornos de prueba.

Clasificación de Sistemas de Control de Versiones

Sistemas Centralizados

Cuentan con un único servidor que almacena el código, los devs se conectan a dicho server para obtener y actualizar el código. Se necesita conexión constante al servidor para poder trabajar.

CVS o SVN por ejemplo.

- ✔ Fácil de administrar.
- ⌦ Depender de un servidor central lo hace vulnerable a fallas y limita el trabajo sin conexión.

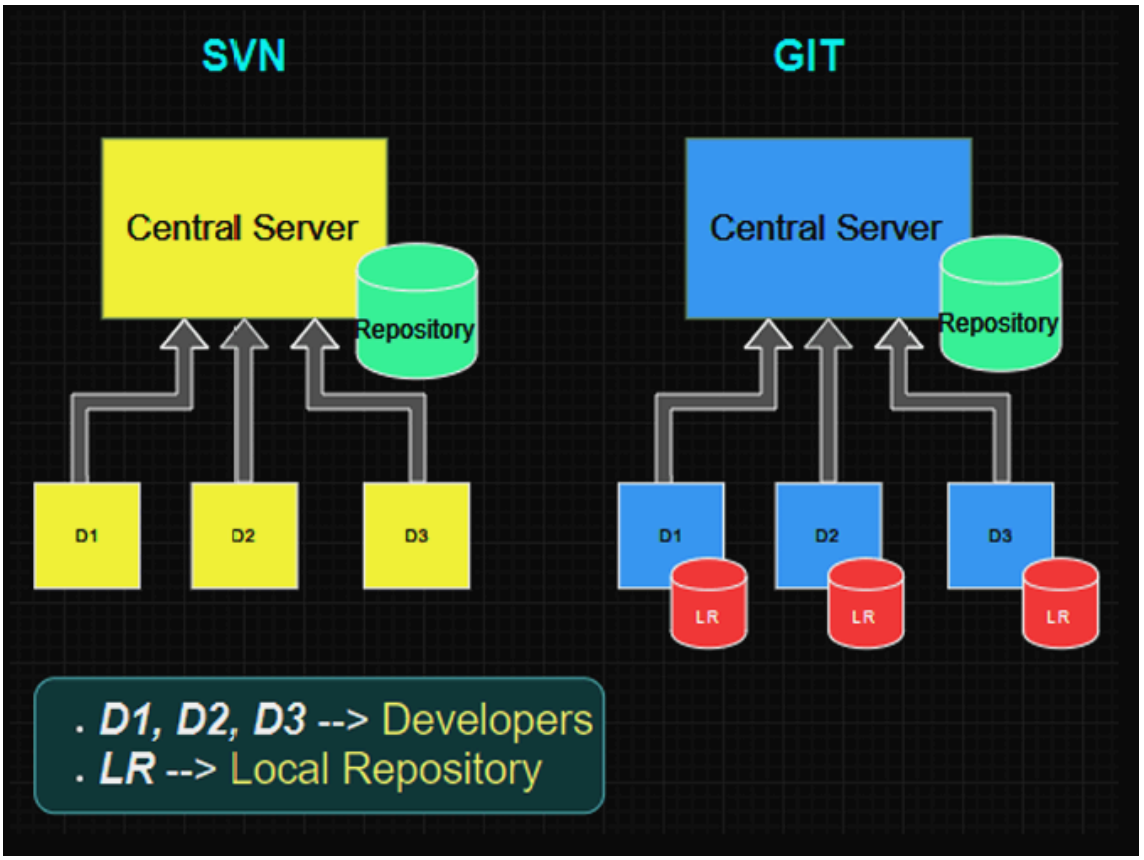
Sistemas distribuidos

Cada dev tiene una copia completa del repositorio, con su historial completo de cambios, así se puede trabajar descentralizado y hacer cambios sin conexión.

Git o Mercurial por ejemplo.

- ✔ Mayor seguridad.
- ✔ Trabajo sin conexión.
- ✔ Colaboración descentralizada.
- ⌦ Mayor espacio en disco, ya que cada usuario tiene una copia completa.

Comparación gráfica



Beneficios del control de versiones

- ✔ Colaboración y trabajo en equipo: Pueden trabajar en paralelo sin interferirse, y los cambios se pueden integrar de manera controlada. Las ramas permiten trabajar nuevas características de forma aislada y fusionarlas al estar listas.
- ✔ Seguimiento histórico de versiones.
- ✔ Facilidad para revertir cambios

Herramientas actuales de control de versiones.

Herramientas más usadas

Herramienta	Ventaja	Desventaja
Git	Sistema distribuido de alto rendimiento. Popular y con soporte en la mayoría de servicios de repositorios.	Curva de aprendizaje inicial.
Mercurial	Similar a Git en funcionalidad distribuida pero con comandos más sencillos y menos errores en la gestión de repositorios.	Menos popular, por lo que tiene menos soporte en herramientas de repositorio.
svn	Aún popular en empresas que prefieren un sistema centralizado y sencillo.	Menos flexible y eficiente que los distribuidos, requiere conexión constante al servidor central.

Repositorios en la nube

- GitHub: Popular por su enfoque social, colaboración en proyectos de código abierto y funcionalidades como seguimiento de issues y los pull request.
- GitLab: Sistema completo de integración y despliegue continuo (CI/CD), y tiene una versión de código abierto.
- Bitbucket: Integra herramientas de Atlassian como Jira, soporta Git y Mercurial. Útil para proyectos privados en empresas.

Las ventajas de usar estos repositorios en la nube son:

- ✔ Acceso remoto y fácil a los proyectos.
- ✔ Funcionalidades de colaboración, CI/CD, y herramientas de gestión de proyectos.
- ✔ Mayor seguridad en la copia de los datos y protección ante pérdidas de datos.

GIT (Básico)

Es un sistema de control de versiones distribuido, diseñado para rastrear y gestionar los cambios en el código fuente de proyectos de software.

Los puntos clave de git son:

- Control de versiones.
- Distribuido.
- Trabajo en equipo.
- Historial detallado.
- Popularidad y soporte.

Como punto clave, cada acción que haces en git lo único que hace es sumar información, nunca resta, por lo que es muy difícil hacer algo que git no pueda remendar.

Comandos básicos (Cosas que puede tomar)

Comando para configurar usuario y mail

```
$ git config --global user.name "John Doe"

$ git config --global user.email johndoe@example.com
```

INIT: Comando para inicializar un repositorio

```
$ git init
```

Clone: Clona un repositorio existente

```
$ git clone https://github.com/libgit2/libgit2
```

.gitignore = archivo para ignorar otros archivos

Para realizar un cambio, primero se tiene que preparar, esta zona de preparación se le dice Staging.

git add index.html agrega el archivo index.html a la zona de preparación.

git commit -m "Actualizacion" la -m es para mandar un mensaje cortito, y lo de las comillas es el mensaje.

Cambia a la rama de destino (a la que quieres incorporar los cambios):

```
git checkout main
```

Realiza el merge desde la rama que deseas incorporar:

```
git merge feature-nueva-funcion
```

Versionado -> no entra en el parcial

El estándar más usado para versionar se llama SemVer (Versionado Semántico). Dado un número de versión MAYOR.MENOR.PARCHE, se incrementa:

1. La versión MAYOR cuando realizas un cambio incompatible en el API,

2. La versión MENOR cuando añades funcionalidad compatible con versiones anteriores, y

3. La versión PARCHE cuando reparas errores compatibles con versiones anteriores.

Hay disponibles etiquetas para prelanzamiento y metadata de compilación como extensiones al formato MAYOR.MENOR.PARCHE.



Etiquetas

Alpha: Versión inicial del desarrollo, no está completa ni estable. Puede contener muchos errores y carece de las funcionalidades principales. La idea es probar ideas, arquitectura y recibir feedback.

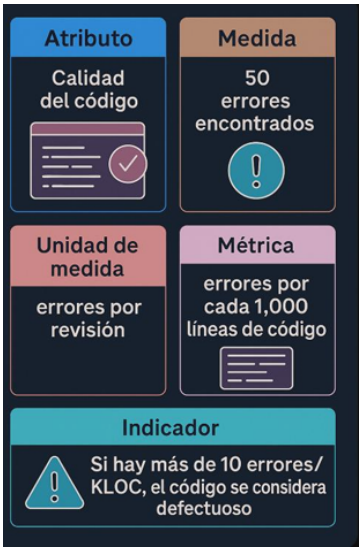
Beta: Mas avanzada que Alpha. El software ya incluye la mayoría de las funcionalidades previstas pero aún puede presentar fallos o problemas de rendimiento. Se libera a un público más amplio para obtener feedback.

Pre-Release: Versión candidata a la final. Esta prácticamente terminado y solo se espera detectar errores o detalles de compatibilidad antes de un lanzamiento estable.

Métricas

Conceptos generales

- **Atributo**: Es una característica o propiedad de un objeto, sistema o entidad que puede ser medible o evaluada.
- **Medida**: Cuantificación numérica de un atributo específico. Las medidas se utilizan para asignar un valor numérico a un atributo con el fin de cuantificarlo y analizarlo.
- **Unidad de medida**: Es la unidad que expresa una medida. Proporciona contexto y significado a los valores numéricos.
- **Métrica:** Medida cuantitativa específica utilizada para evaluar, analizar o comparar el desempeño, calidad o características de un objeto o sistema.
- **Indicador**: Métrica específica utilizada para señalar una tendencia, un estado o una condición en particular. Estos indicadores a menudo se utilizan para tomar decisiones o para señalar la necesidad de tomar medidas correctivas.



Ventaja del uso de métricas

- Toma de decisiones basadas en datos.
- Control de calidad.
- Mejora continua.
- Gestión de rendimiento.
- Evaluación de cumplimiento de objetivos (Muy usada por la metodología Scrum, donde evalúan por historias de usuario).

Mal uso

- Exceso de métricas.
- Manipulación incorrecta.
- Falta de contexto.
- Enfoque en cantidad en lugar de calidad.
- Resistencia al cambio.

Modelos de métricas sugeridos

Modelo que establece una jerarquía lógica entre objetivos (goal), preguntas (question) y métricas (metric).

Su enfoque se basa en objetivos claros y traducirlos en preguntas medibles, para luego diseñar métricas que permitan responderlas objetivamente.

Ampliamente usado para evaluar la calidad de software, mejora de procesos, y toma de decisiones basada en métricas.

ISO / IEC 25010

Estándar internacional que define características de calidad del producto de software y su evaluación. Se enfoca en la calidad del producto según 8 atributos de la calidad de software. Se usa para evaluar formalmente productos de software, auditorías, certificaciones de calidad.

CMMI

Modelo de madurez de procesos que define niveles de mejora continua para la organización.

Agile Metrics

Conjunto de métricas que se usan en metodologías ágiles como Scrum o Kanban para evaluar entrega de valor y eficiencia en equipo.

ITIL Metrics

Conjunto de métricas derivadas del marco ITIL para la gestión de servicios de TI.